



Machine Learning Systems, Spring 2026

Course Project Phase 2: Agentic Optimization of a LoRA Operator

Deadline: May 12, 2026, 8:00 AM

Evaluation Device: NVIDIA GeForce RTX 3090

In Phase 1, you built an agentic profiling tool that could inspect GPU properties and reason about performance.

In Phase 2, you will build an **optimization agent** for a LoRA-style operator. The goal is not to hand in a single manually written kernel. The goal is to build an **agent system** that can iteratively generate, test, profile, and improve a CUDA implementation.

1. Task

Your target operator is

$$Y = WX + A(B^T X)$$

where:

$$W \in \mathbb{R}^{d \times d}, X \in \mathbb{R}^{d \times d}, A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{d \times r}, r = 16.$$

All tensors are stored as **.pt files** and can be loaded with `torch.load`.

All hidden evaluation tensors use **float32**.

Shape range

Instead, the hidden evaluation will choose several sizes with:

- **hidden dimension \$d\$** selected from a range in [3584, 4608].

- for this project, use the following public range:

$$d \in [3584, 4608]$$

For evaluation, we will choose several test cases with d inside this range.

You should therefore design your agent and your generated CUDA code to handle **multiple sizes** in this interval rather than overfitting to one exact matrix shape.

2. What you must submit

Your submission must contain at least:

- `run.sh`

During execution, your system must maintain a file:

- `optimized_lora.cu`

Submission contract

- The evaluation system will enter the **submission root directory**
- It will run:

```
bash run.sh
```

- It will later read:

```
./optimized_lora.cu
```

from the **same directory**.

Time budget

Your agent may run for **up to 30 minutes**.

At timeout or normal completion, the evaluation system will read the **final** `optimized_lora.cu` in the submission root and benchmark that file using the official harness.

Therefore:

- you must **always maintain a latest compilable version** of `optimized_lora.cu`
- do not wait until the very end to produce your first working version

3. What your agent is expected to do

Your submission must be an **actual optimization agent**.

A valid agent should be able to:

- generate candidate CUDA implementations
- compile and test them
- benchmark them
- compare alternatives
- iteratively improve the current best implementation
- keep the current best version in `optimized_lora.cu`

Because the hidden evaluation tensors are not exposed to your agent during official testing, your agent should generate its **own synthetic test tensors** within the public size range and use them for local search.

Important clarification

This project is **not** asking you to hand in a single static kernel.

The course staff expect a real agentic workflow rather than a one-shot, fixed solution.

4. Official evaluation environment

The official evaluation environment is:

- **GPU:** NVIDIA GeForce RTX 3090
- **OS:** Ubuntu 22.04.4 LTS
- **Python:** 3.10.12
- **PyTorch:** 2.3.0a0+6ddf5cf85e.nv24.04

- **CUDA toolkit:** 12.4
- **GCC:** 11.4.0

`nvcc` is available from the CUDA toolkit installation.

When needed, you may also rely on the standard PyTorch extension toolchain.

5. Official Python harness

The course staff will use a Python harness to:

1. load hidden `W.pt`, `X.pt`, `A.pt`, `B.pt`
2. compile `optimized_lora.cu`
3. call the exported CUDA implementation
4. compare its output with a standard PyTorch reference
5. benchmark runtime
6. compute speedup against the standard PyTorch implementation
7. write the result to `result.out`

A simplified reference version is shown below.

```
import torch
from pathlib import Path
from torch.utils.cpp_extension import load

def load_inputs(base_dir: str):
    base = Path(base_dir)
    W = torch.load(base / "W.pt", map_location="cpu").contiguous().cuda()
    X = torch.load(base / "X.pt", map_location="cpu").contiguous().cuda()
    A = torch.load(base / "A.pt", map_location="cpu").contiguous().cuda()
    B = torch.load(base / "B.pt", map_location="cpu").contiguous().cuda()
    return W, X, A, B

def reference_impl(W, X, A, B):
    with torch.no_grad():
        return W @ X + A @ (B.transpose(0, 1).contiguous() @ X)

def build_module(cu_path: str):
```

```

module = load(
    name="optimized_lora_ext",
    sources=[cu_path],
    verbose=False,
    extra_cuda_cflags=["-O3"],
    with_cuda=True,
)
return module

def check_correctness(y, y_ref):
    diff = (y - y_ref).float()
    max_abs_err = diff.abs().max().item()
    rel_l2_err = (diff.norm() / (y_ref.float().norm() + 1e-12)).item()
    passed = torch.allclose(y, y_ref, rtol=1e-4, atol=1e-4)
    return passed, max_abs_err, rel_l2_err

def benchmark(fn, W, X, A, B, warmup=10, iters=50):
    for _ in range(warmup):
        _ = fn(W, X, A, B)
    torch.cuda.synchronize()

    times = []
    for _ in range(iters):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        _ = fn(W, X, A, B)
        end.record()
        torch.cuda.synchronize()
        times.append(start.elapsed_time(end)) # milliseconds

    times.sort()
    return times[len(times) // 2]

def main():
    input_dir = "./hidden_inputs"
    cu_path = "./optimized_lora.cu"
    result_path = "./result.out"

    W, X, A, B = load_inputs(input_dir)
    module = build_module(cu_path)

    with torch.no_grad():
        y_student = module.forward(W, X, A, B)
        y_ref = reference_impl(W, X, A, B)

```

```

passed, max_abs_err, rel_l2_err = check_correctness(y_student, y_ref)

if passed:
    student_ms = benchmark(module.forward, W, X, A, B)
    torch_ms = benchmark(reference_impl, W, X, A, B)
    speedup = torch_ms / student_ms
else:
    student_ms = None
    torch_ms = None
    speedup = 0.0

with open(result_path, "w") as f:
    f.write(f"correct: {passed}\n")
    f.write(f"max_abs_err: {max_abs_err}\n")
    f.write(f"rel_l2_err: {rel_l2_err}\n")
    f.write(f"student_median_ms: {student_ms}\n")
    f.write(f"torch_median_ms: {torch_ms}\n")
    f.write(f"speedup: {speedup}\n")

if __name__ == "__main__":
    main()

```

You are encouraged to use a compatible local harness inside your own agent workflow to reduce environment mismatch.

6. Required interface for `optimized_lora.cu`

Your final `optimized_lora.cu` must be:

- a single file
- self-contained
- directly compilable by the official Python harness
- able to export a callable entry point

The expected callable interface is:

```

torch::Tensor forward(torch::Tensor W,
                      torch::Tensor X,
                      torch::Tensor A,

```

```
torch::Tensor B);
```

and the module must expose it via `PYBIND11_MODULE(...)`, so that the harness can call:

```
module.forward(W, X, A, B)
```

Allowed dependencies

You may use:

- standard CUDA headers
- standard C/C++ library headers
- standard PyTorch extension headers already available in the system environment

You may **not** depend on extra submission-side source files beyond `optimized_lora.cu`.

That means no additional required files such as:

- `.cu`
- `.cuh`
- `.h`
- `.cpp`

outside the final `optimized_lora.cu`.

7. Dedicated rules you must read carefully

7.1 Input and output format

- Hidden inputs are stored as `.pt` files
- They are loaded with `torch.load`
- Hidden evaluation tensors use `float32`
- The tested operator is:

$$Y = WX + A(B^T X)$$

- The hidden dimension d is **not fixed**

- Evaluation uses multiple sizes with $d \in [3584, 4608]$
- The low-rank dimension is fixed at **16**
- Your generated CUDA implementation must accept the provided tensors and return the output tensor

7.2 Scoring strategy and standards

Correctness is a **hard requirement**.

A submission that does **not** pass correctness checking will **not** receive performance credit.

Correctness is checked against the PyTorch reference:

$$Y_{\text{ref}} = WX + A(B^T X)$$

using:

```
torch.allclose(Y_student, Y_ref, rtol=1e-4, atol=1e-4)
```

We also record:

- `max_abs_err`
- `rel_l2_err`

For submissions that pass correctness, the final score is:

- **70% Speedup**
- **30% Agent Implementation / Engineering Methodology**

Speedup

Speedup is computed as:

$$\text{speedup} = \frac{\text{median runtime of standard PyTorch implementation}}{\text{median runtime of your CUDA implementation}}$$

Runtime measurement uses:

- **warmup first**
- CUDA events

- median latency over repeated runs

Agent Implementation / Engineering Methodology

This part rewards submissions that genuinely implement an optimization agent, including factors such as:

- iterative improvement workflow
- candidate generation and comparison
- use of benchmarking/profiling for decision making
- code organization and reproducibility
- overall engineering quality of the agent system

7.3 You must use your own API key

If your agent relies on external model APIs, you must use **your own API key**.

The course staff will **not** provide API credits for you.

You should design your system so that your own API key can be supplied safely and cleanly, for example through environment variables or local configuration used by your submission.

7.4 Strictly prohibited

The following are prohibited:

1. **Submitting only a static kernel**
 - This project requires an **agent**, not just a fixed hand-written final kernel
2. **Hardcoding the final CUDA code inside the agent**
 - Do not embed a prewritten final `optimized_lora.cu` as a fixed literal/template/string inside your agent and simply dump it at runtime
 - Your agent should actually perform optimization rather than merely revealing a hidden final answer
3. **Depending on extra source files for the final measured implementation**
 - The final measured implementation must be the single-file `optimized_lora.cu`
4. **Breaking the official I/O contract**
 - The evaluation system runs `bash run.sh` in the submission root and reads `./optimized_lora.cu` from the same directory

8. Practical advice

A strong submission will usually:

- generate candidate CUDA code variants
- compile and test them automatically
- verify correctness against a local PyTorch reference
- benchmark repeatedly after warmup
- keep the current best valid implementation in `optimized_lora.cu`
- avoid overfitting to one exact matrix size

9. Summary

In this project, you are building an **agentic CUDA optimization system** for a LoRA operator.

Your agent should:

- run from `bash run.sh`
- iteratively improve CUDA code
- always keep a valid `optimized_lora.cu`
- produce a single-file, self-contained final CUDA implementation
- optimize for the operator

$$Y = WX + A(B^T X)$$

over multiple hidden test sizes in [3584, 4608].

Only correct implementations are ranked, and among correct submissions the final evaluation is based on:

- **70% speedup**
- **30% agent implementation / engineering methodology**